

THE MATCOM MINIX

Informe Técnico

Estudiante 1: Miguel Zamora

Grupo: [COMPLETAR]

Estudiante 2: [COMPLETAR SI APLICA]

Grupo: [COMPLETAR]

May 4, 2026

1 Introducción

Este proyecto se desarrolló en el contexto de la asignatura Sistemas Operativos con el objetivo de estudiar, desde la práctica, principios de concurrencia, planificación y sistema de archivos sobre una base real: MINIX. A diferencia de ejercicios aislados, la metodología usada exige intervenir código productivo del sistema, recompilar, validar y documentar resultados.

Los objetivos generales fueron: (1) preparar el entorno de trabajo en MINIX, (2) realizar una modificación visible del sistema (`/etc/motd`), (3) depurar un bug real en la capa de compatibilidad `pthread_*`, (4) implementar un comando de usuario (`tree`) y (5) extender el scheduler para distinguir procesos CPU-bound y aplicar penalización gradual.

Este informe documenta específicamente el estado actual del repositorio `minix` y los cambios efectivamente registrados en Git, trazándolos contra el *Cronograma de hitos* entregado en `orientation/Cronograma de hitos.pdf`.

2 Desarrollo y resultados por componente

2.1 Trazabilidad de hitos vs cambios reales en el repositorio

Hito	Requisito del cronograma	Fecha/commit	Evidencia en repo
1	Entorno listo + primera modificación visible (<code>/etc/motd</code>)	2026-04-16, 567869d6f (+ ajustes en 03b0c367a, 06d57c478, 60b69581b)	<code>etc/motd</code> modificado con mensaje personalizado FMATCOM/UH.
2	Corrección bug <code>pthread_mutex_trylock</code> + comando <code>tree</code>	2026-04-18, 35634acc7 y fixes posteriores; 2026-04-27, 67eeae1b7	Archivos nuevos <code>bin/tree/</code> ; integración en <code>bin/Makefile</code> ; fix de recursión en <code>pthread_compat.c</code> .

2.2 Instalación y configuración de VirtualBox y MINIX

La evidencia directa de instalación (versión exacta de VirtualBox, parámetros de VM, incidentes de boot/red) no se almacena en este repositorio, por lo que no puede reconstruirse de forma determinista solo desde Git. Sí existe evidencia indirecta de que el entorno quedó operativo, dada la secuencia de commits funcionales sobre componentes del sistema y de espacio de usuario.

Para la entrega final, se recomienda completar esta sección con: versión de VirtualBox, versión exacta de MINIX usada en la VM, RAM, núcleos, disco, red, y cualquier incidencia operativa observada durante la instalación.

2.3 Personalización del mensaje de bienvenida

Ruta modificada: `/etc/motd` (en el repositorio: `etc/motd`).

El primer hito introdujo una personalización explícita del mensaje de bienvenida con referencia a la Facultad de Matemática y Computación (UH), cumpliendo el requisito de modificación visible del sistema al iniciar sesión.

Listing 1: Contenido actual relevante de `etc/motd`

```
*=====*
```

Bienvenido a esto que dice ser un	*
Sistema Operativo.	*
Manicomio de Matematica y Computacion	*
Colina del Terror de La Habana (UH)	*

```
*=====*
```

2.4 Depuración de un bug en pthread

Síntoma esperado según orientación: bloqueo en la segunda llamada a `pthread_mutex_trylock`.

Análisis: la capa `pthread_*` en MINIX delega en `mthread_*`. En `minix/lib/libmthread/pthread_compat` la implementación de `pthread_mutex_trylock` llamaba recursivamente a sí misma, provocando fallo lógico severo.

Corrección aplicada (mínima):

Listing 2: Diff funcional del fix de pthread

```
- return pthread_mutex_trylock(mutex);
+ return mthread_mutex_trylock(mutex);
```

El cambio quedó registrado en el commit `67eeae1b7`. Esta corrección concuerda con el Hito 2 y con la relación requerida `pthread_*` (compatibilidad) → `mthread_*` (implementación real).

2.5 Implementación del comando tree

La implementación se integró como comando de usuario mediante:

- alta de `bin/tree/tree.c`,
- alta de `bin/tree/Makefile`,
- inclusión de `tree` en `bin/Makefile`.

Diseño del algoritmo: recursivo explícito (`print_tree(path, depth)`), adecuado por simplicidad y correspondencia directa entre nivel de recursión e indentación visual.

Llamadas al sistema usadas:

- `opendir/readdir/closedir` para iteración de directorios.
- `lstat` para obtener metadatos sin seguir enlaces simbólicos.

Prevención de ciclos: el código evita descender por symlinks mediante `!S_ISLNK(entry_stats.st_mode)`, cumpliendo el requisito obligatorio de la guía.

Listing 3: Fragmento clave de `bin/tree/tree.c`

```
int check = lstat(fullpath, &entry_stats);
if (!(check == -1) && S_ISDIR(entry_stats.st_mode) &&
    !S_ISLNK(entry_stats.st_mode)) {
    print_tree(fullpath, depth + 1);
}
```

2.6 Penalización por uso intensivo de CPU

Análisis previo y puntos intervenidos:

- Agotamiento de quantum: `do_noquantum()` en `schedule.c`.
- Balance periódico: `balance_queues()`.
- Estado por proceso: `struct schedproc` en `schedproc.h`.

Diseño implementado:

- Ventana temporal ya existente del balanceador (`BALANCE_TIMEOUT = 5s`).
- Umbral CPU-bound: `CPU_BOUND_QUANTUM_THRESHOLD = 3`.
- Penalización máxima: `MAX_PENALTY_LEVEL = 2`.

Cambios de datos por proceso:

Listing 4: Campos añadidos en `schedproc.h`

```
unsigned base_priority;
unsigned quantum_count;
unsigned penalty_level;
```

Cambios de lógica:

- `do_noquantum()` incrementa `quantum_count` para procesos no de sistema.
- `do_start_scheduling()` inicializa `base_priority`, `quantum_count` y `penalty_level`.
- `balance_queues()` aplica penalización si `quantum_count >= 3`, recuperación gradual si `quantum_count == 0`, y reinicia contador por ventana.

La modificación corresponde al Hito 3 y materializa la política solicitada en `Orientación del proyecto.pdf`.

3 Resultados globales y discusión

Desde la perspectiva del repositorio, los tres hitos del cronograma están cubiertos por cambios concretos, versionados y trazables. El Hito 1 queda evidenciado por la personalización de `motd`; el Hito 2 por la implementación/integración de `tree` y la corrección de `pthread_mutex_trylock`; y el Hito 3 por la extensión del scheduler con política de penalización CPU-bound por ventanas.

La principal limitación metodológica de este análisis es que se basa en evidencia estática de código y commits (no en ejecución dentro de la VM MINIX en este entorno). Por ello, el cierre formal del informe académico debe incluir anexos de ejecución (salidas del programa de prueba `pthread`, pruebas de `tree` en `.`, `/usr`, `../`, y evolución de prioridad en escenarios CPU-bound vs interactivo).

4 Conclusiones

El estado actual del repositorio es coherente con el *Cronograma de hitos*: las funcionalidades objetivo fueron implementadas y quedaron reflejadas en commits fechados dentro del período planificado. Técnicamente, el trabajo aborda cuatro núcleos de aprendizaje de la asignatura: modificación del sistema base (`motd`), depuración de concurrencia (`pthread`), desarrollo de herramienta de sistema de archivos (`tree`) y ajuste de política de planificación (scheduler).

Como mejora recomendada para robustecer la entrega final: agregar un conjunto mínimo de scripts o logs reproducibles de validación experimental para los apartados 2.3, 2.4 y 2.5.

5 Referencias consultadas

1. orientation/Orientación del proyecto.pdf
2. orientation/Cronograma de hitos.pdf
3. orientation/THE MATCOM MINIX_Informe técnico.pdf
4. Código fuente del repositorio MINIX (archivos `etc/motd`, `bin/tree/*`, `minix/lib/libmthread/pthread_com`, `minix/servers/sched/*`).
5. Historial Git del proyecto (commits entre `567869d6f` y `7c92edd4e`).

6 Declaración de uso de IA

Se utilizó IA generativa como apoyo para: (1) sintetizar requisitos de los documentos PDF, (2) estructurar y redactar técnicamente el informe en $\text{\LaTeX}$, y (3) organizar la trazabilidad entre hitos y commits del repositorio. Las decisiones de implementación documentadas, así como los cambios de código referenciados, provienen del historial real del repositorio analizado.

7 Anexos (si aplica)

A. Línea de tiempo resumida de commits del proyecto

- 2026-04-16: `567869d6f` (*1er hito*) – modificación inicial de `motd`.
- 2026-04-18: `35634acc7` (*2do hito: tree command*) + fixes posteriores en `tree` y `motd`.
- 2026-04-27: `67eeae1b7` (*fix pthread*).
- 2026-05-04: `e3e48819b` (*scheduler CPU-bound penalty*) + commits de limpieza de `.gitignore`.